

ROS2

DAY 5 – Manual 1

Actions – Fundamentals & Setup

Objective

By the end of this manual, you will be able to:

1. Understand ROS2 Actions and why they exist
 2. Know the difference between Topics, Services, and Actions
 3. Understand the structure of .action files
 4. Create a ROS2 interface package for actions
 5. Configure CMakeLists.txt and package.xml properly
 6. Build the workspace so ROS2 generates the action code
 7. Verify the generated action interfaces
-

Step 0 — Pre-Requisites

Before starting:

- ROS2 installed (Foxy / Humble / Iron / Rolling)
 - A workspace: ~/ros2_ws
 - Basic knowledge of services (Day 04)
 - Python 3 installed
 - Terminal usage basics
-

Step 1 – What Are ROS2 Actions?

Actions are used when:

✓ The task takes time

Examples:

- Move robot arm to a position
- Navigate robot to a goal
- Download a file
- Compute something long (Fibonacci, PID tuning, mapping...)

✓ Client needs:

- Send a goal
- Receive continuous feedback
- Wait for final result

🔄 Action communication flow:

Client ----- sends goal -----> Server
Client <----- feedback <----- Server
Client <----- result ----- Server

Step 2 – Action vs Service vs Topic

Feature	Topic	Service	Action
Direction	Publish/Subscribe	Request/Response	Goal → Feedback → Result
Duration	Continuous	Instant	Long-running
Cancel support	✗	✗	✓
Feedback	✗	✗	✓
Example	Sensor data	Addition	Robot navigation

Simple rule:

- Use Topics → when publishing data continuously
 - Use Services → when you need an instant answer
 - Use Actions → when the task takes time + needs feedback
-

Step 3 – Anatomy of an .action File

A .action file has three sections, separated by ---.

Example: Fibonacci.action

```
int32 order          # goal
---
int32[] sequence     # result
---
int32[] partial      # feedback
```

Meaning:

- Goal → what the client asks
 - Result → what the server returns when complete
 - Feedback → updates while running
-

Step 4 – Create an Interface Package for Actions

We will create:

- ✂ Package: action_pkg
 - 📄 Action file: Fibonacci.action
-

4.1: Create the package

```
cd ~/ros2_ws/src
ros2 pkg create action_pkg --build-type ament_cmake
```

This creates a C++ interface package (required for actions).

4.2: Create the action folder

```
cd action_pkg
mkdir -p action
```

4.3: Create the action file

nano action/Fibonacci.action

Paste:

```
int32 order
---
int32[] sequence
---
int32[] partial
```

Save → CTRL+O → ENTER → CTRL+X

Step 5 – Update package.xml

Open:

```
nano package.xml
```

Add these inside <package>:

```
<buildtool_depend>ament_cmake</buildtool_depend>
<build_depend>roslint_default_generators</build_depend>
<exec_depend>roslint_default_runtime</exec_depend>
<member_of_group>roslint_interface_packages</member_of_group>
```

Why?

- roslint_default_generators → generates action code
- roslint_default_runtime → allows other packages to use it

- `ament_cmake` → required build tool
 - `member_of_group` → tells ROS this package contains interfaces
-

Step 6 – Update CMakeLists.txt

Open:

```
nano CMakeLists.txt
```

Ensure it contains:

```
cmake_minimum_required(VERSION 3.8)
project(action_pkg)

find_package(ament_cmake REQUIRED)
find_package(rosidl_default_generators REQUIRED)

rosidl_generate_interfaces(${PROJECT_NAME}
  "action/Fibonacci.action"
)

ament_export_dependencies(rosidl_default_runtime)

ament_package()
```

Explanation (simple):

- `find_package` → load generator modules
 - `rosidl_generate_interfaces` → generate action code
 - `ament_export_dependencies` → allow other packages to import the action
 - `ament_package()` → finalize package generation
-

Step 7 – Build

```
cd ~/ros2_ws
colcon build --packages-select action_pkg
source install/setup.bash
```

Step 8 – Verify

List action types generated:

```
ros2 interface show action_pkg/action/Fibonacci
```

Should show:

```
int32 order
---
int32[] sequence
---
int32[] partial
```

Step 9 – Concepts Recap

- ✓ Actions are used for long tasks
 - ✓ They support feedback + cancel + result
 - ✓ .action file defines Goal / Result / Feedback
 - ✓ Interface package must be built BEFORE Python nodes
 - ✓ CMakeLists + package.xml must include correct dependencies
-

Manual 1 Complete!

Next → Manual 2: Implement Action Server & Action Client (Python)

You will get:

- Complete code
 - Line-by-line explanation
 - Execution steps
 - Debugging tips
-